Exercise on Mutation Testing

Week 5: Mutation Testing

Below are example solutions to the exercises. While these address the requirements, you are encouraged to explore alternative approaches and solutions.

Your creativity and problem-solving strategies are valued :)

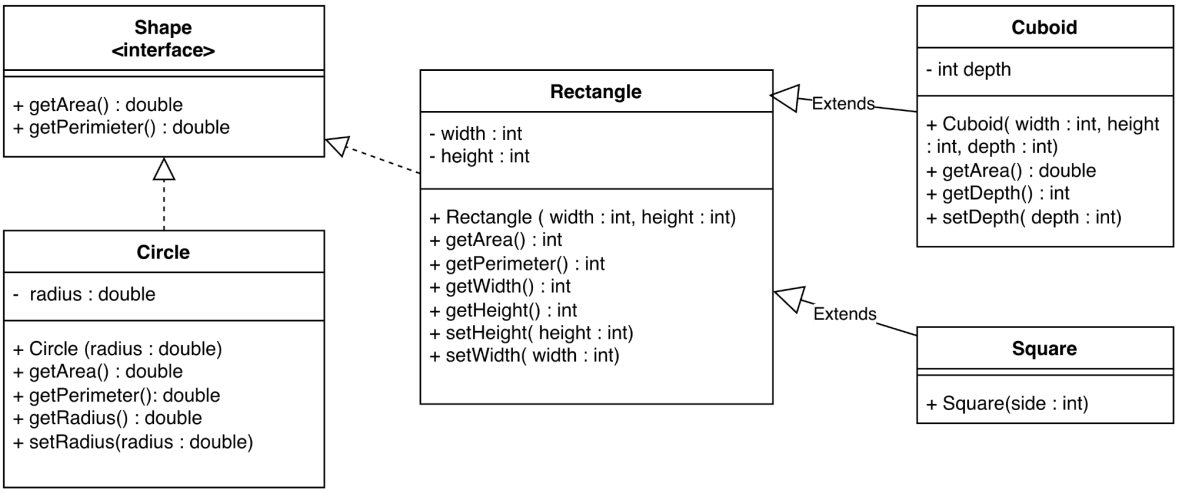
(Solutions for the optional exercises are not provided)

Setup the project:

1. Install Maven. Search Youtube for guides in your preferred OS.
    a. Make sure your environmental variables are defined.
    b. Run mvn -v on your terminal to see that Maven is working.
2. Download and unzip "QAT_Week5_Exercise_Mutation" from Canvas.
3. Import the Java project to your IDE (e.g., Visual Studio Code)

Part 1: Understanding the Code and Designing Tests

1. The diagram below shows the different classes and their relationships



2. Generate 10 test cases for the classes below.

See link for the test cases details (src → test):

T1: shouldCreateValidCircle (CircleTest)

T2: shouldComputeCorrectArea (CircleTest)

T3: shouldComputeCorrectPerimeter (CircleTest)

T4: shouldThrowExceptionForInvalidRadius (CircleTest)

T5: testValidCuboidAreaAndGetters (CuboidTest)

T6: testInvalidDepthThrowsException (CuboidTest)

T7: testValidRectangleProperties (RectangleTest)

T8: testInvalidHeightThrowsException (RectangleTest)

T9: testInvalidWidthSetterThrowsException (RectangleTest)

T10: testSquareInitializationAndBehaviour (SquareTest)

3. Manually create 10 mutants and add them to the table below:

ID	Mutation Operator	Class	LoC	Mutant
M1	Relational Operator Replacement (ROR)	Circle	17	if (radius < 0) instead of if (radius <= 0)
M2	Arithmetic Operator Replacement (AOR)	Circle	25	Math.PI * Math.pow(radius, 1) instead of Math.pow(radius, 2)
M3	Arithmetic Operator Replacement (AOR)	Circle	28	return 2 * Math.PI / this.radius instead of return 2 * Math.PI * this.radius
M4	Statement Replacement (STR)	Circle	32	return "Circle: r:" instead of return “Circle: r:” + radius
M5	Relational Operator Replacement (ROR)	Cuboid	24	if (depth < 0) instead of if (depth <= 0)
M6	Arithmetic Operator Replacement	Cuboid	17	int areaFront = (int) super.getArea() / 2; instead of int areaFront = (int) super.getArea() *

	(AOR)			2;
M7	Relational Operator Replacement (ROR)	Rectangle	24	if (height < 0) instead of if (height <= 0)
M8	Arithmetic Operator Replacement (AOR)	Rectangle	13	return this.height + this.width; instead of return this.height * this.width;
M9	Arithmetic Operator Replacement (AOR)	Rectangle	17	return (2 * this.height) - (2 * this.width); instead of return (2 * this.height) + (2* this.width)
M10	Argument Replacement (ARG)	Square	6	super(side, 1) instead of super(side, side)

4. Run your test cases against the mutants. Fill in the mutation matrix and calculate the mutation score.

MutationScore = killedMutants / totalMutants = 9/10 = 0.9

	M1	**M2**	**M3**	**M4**	**M5**	**M6**	**M7**	**M8**	**M9**	**M10**
T1										
T2		x								
T3			x							
T4	x									
T5						x				
T6					x					
T7								x	x	
T8							x			

T9										
T10										x

5. Write down your reflections on the points below:

    a. Did you use a variety of operators? Were some harder/easier to apply than others?

A good variety of mutation operators was used, including Relational Operator Replacement (ROR), Arithmetic Operator Replacement (AOR), Statement Replacement (STR), and Argument Replacement (ARG).
AOR and ROR mutants were the easiest to apply, as they involved simple changes to arithmetic expressions or relational operator.
STR and ARG mutants were slightly harder, especially in small classes like Square, where there are few mutation locations.

    b. Was there a type of mutant that was harder / easier to kill?

Relational Operator Replacement (ROR) mutants (e.g., M1, M5, M7) were harder to kill, because they only affect boundary conditions and require specific edge-case tests (such as zero values).
In contrast, Arithmetic Operator Replacement (AOR) mutants were easier to kill, as incorrect calculations led to clearly wrong results that were quickly detected by existing tests.

    c. Was there a test case that is very good at killing multiple mutants?

The only test that killed more than one mutant was T7, because it checks multiple properties (area, perimeter, etc).

Part 2: Automating Mutation Testing with PIT

1. PIT should be configured in your project's pom.xml file.
2. In the terminal (project root), run the command (https://pitest.org/quickstart/maven/):

```
mvn test-compile org.pitest:pitest-maven:mutationCoverage
```

3. Check the mutation report in folder: target > pit-reports > index.html

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
4	75%	41/55	85%	39/46	95%	39/41

Breakdown by Package

Name	Number of Classes	Line Coverage		Mutation Coverage		Test Strength	
exercise_shapes	4	75%	41/55	85%	39/46	95%	39/41

Overall, mutation coverage (85%) was higher than line coverage (75%), indicating that although not all lines were executed, many executed lines were tested effectively against faults. This shows that high line coverage alone was not required to achieve strong mutation coverage, reinforcing the idea that test quality matters more than quantity.

Arithmetic-related mutants were handled very well across all classes: both the MathMutator (100% killed) and PrimitiveReturnsMutator (100% killed) indicate that calculations and return values in classes such as Rectangle, Circle, and Cuboid were thoroughly tested. This explains why mutation coverage remained high despite moderate line coverage.The ConditionalsBoundaryMutator achieved only 75%, indicating that some classes lacked sufficient edge-case tests (e.g., zero or boundary values). Similarly, the VoidMethodCallMutator had the lowest effectiveness (40% killed, with several no-coverage cases).